

Self-Assessment

Advanced Deep Learning D7047E

Name: **Adesijibomi Aderinto**

ILO	Description: <i>CNN and Deeper</i>	Status
	Learning Outcomes	Completed
	- Recap: Understand the basics of machine learning and NN - Have deeper insights about the backpropagation algorithm applied to NN and CNN - Describe several CNN architectures - Understand the difference between various common practices (pooling, padding, regularization)	

1. Multiple layers stacked as follows:

- Convolutional Layers: Detect local patterns using filters/kernels.
- Pooling Layers: Downs ample spatial dimensions to reduce computation and control overfitting.
- Fully Connected Layers: Final layers for classification.
- Padding: Add extra pixels to maintain image size.
- Stride: Controls the step of the convolution filter.

2. Transfer Learning:

- Transfer learning is a deep learning technique where a model developed for one task is reused as the starting point for a model on a second task. It's especially useful when you have limited data for your target problem but can leverage knowledge from a model trained on a large dataset
- Use pretrained CNNs like VGG, ResNet, Inception as feature extractors.
- Fine-tune on custom datasets (great for small datasets).
- Freezing the early layers to preserve learned features.

3. Data Augmentation:

Techniques like rotation, shifting, flipping, zooming, to identify more features from the images,

It helps improves generalization to unseen data better.

4. Regularization:

is a set of techniques used to prevent overfitting in models, it is applied during training to prevent model from performing poorly on unseen data, selecting only the important features while penalizing the less important features.

- I. L1 Least absolute shrinkage regression and selection operator add a penalty term to the loss function

$$\text{Loss} = \text{MSE (Mean Squared Error)} + \lambda \sum |w_i|$$

where λ is a hyperparameter that determines the strength of the regularization.

- II. L2 Ridge regression adds a penalty equal to the square of the magnitude of the model coefficients to the loss function, it discourages large weights but does not force them to zero.

$$\text{Loss} = \text{MSE (Mean Squared Error)} + \lambda \sum w_i^2$$

5. Batch normalization:

Is a technique to normalize the inputs of each layer so that they have a mean of 0 and a standard deviation of 1, typically applied during training of deep neural networks. It speeds up training, improve performance, improve gradient flow. Usually used before activation function.

6. Dropout:

Regularization technique that randomly "drops" (sets to zero) a fraction of neurons during training. It prevents overfitting and improves generalization. However, dropout cannot be used during model evaluation, maybe less desirable with smaller models, Often, BatchNorm alone is enough regularization, using both maybe less effective.

7. Confusion Matrix:

Accuracy measures the correctness of the model, usually good for balanced datasets, unskewed datasets.

$$\frac{TP + TN}{TP + TN + FP + FN}$$

Precision: measure the predicted positives.

$$\frac{TP}{TP + FP}$$

Recall: How many positives were predicted correctly

$$\frac{TP}{TP + FN}$$

F1 Score: Harmonic mean of precision and recall, Imbalanced datasets where both false positives and false negatives matter.

8. Max Pooling:

Down sampling operation commonly used in CNNs to reduce the spatial dimensions (width and height) of the feature maps. It helps reduce computation; it prevents overfitting by acting as a form of regularization. Average pooling is also a variant that can be applied

ILO	Description: <i>CNN Enhancements</i>	Status
	Learning Outcomes	Completed
	- Understand deeper architectures, such as ResNet and Inception - Generalize to other, more recent tweaks.	

ResNet is a deep learning trick that allow deep learning model with large depth or many layers to train using residuals learning, i.e.

$$F(x) = H(x) - x$$

Where $H(x)$ is the expected output and x is the input.

- Solves vanishing gradient problem
- Allows very deep networks (up to 152+ layers)
- Improves accuracy on ImageNet and other benchmarks
- Faster to converge
- Used in image classification, object detection, image segmentation.

Inception:

Inception is a deep convolutional neural network architecture designed to improve efficiency and accuracy by combining multiple convolutional filters of different sizes in parallel within a single layer, called an Inception module. This allows the network to capture features at various scales simultaneously. It uses 1x1 convolutions for dimensionality reduction, reducing computational cost while preserving important information. Inception networks (like GoogLeNet) are known for being both deep and computationally efficient, helping achieve strong performance on image recognition tasks.

Convolution operation: Can be described as how a small matrix called a kernel or filter is applied to an image to produce a feature map

Valid Kernel can be described as a type of kernel where no padding is applied, i.e. Input = 5x5, Kernel = 3x3 → Output = 3x3, but kernel type is not good for capturing edge information

Same Kernel can be described as kernel where zero padding is added to the kernel to maintain same input size with the output size after convolution. i.e. Input = 5x5, Kernel = 3x3 → Output = 5x5 (with 1-pixel padding)

Full Kernel: Adds maximum padding, allowing the kernel to overlap the input as much as possible, even with partial overlaps. Output is larger than the input. i.e. Input = 5x5, Kernel = 3x3 → Output = 7x7

Correlation: Applies a kernel/filter to an image by sliding it across and computing the dot product at each position.

Convolution: Like correlation, but the kernel is flipped both horizontally and vertically before applying the dot product.

Backpropagation: We compute the gradient of the loss with respect to the filter weights (kernels) and loss with respect to the input feature maps, that way what kinds of features to detect, like edges, corners, or textures. During backpropagation If the input to ReLU was positive, gradient flows back. If the input was negative, gradient becomes zero (no update from that neuron). Pls note the max pooling layer has not learning parameters, so no weight update happens here.

ILO	Description: <i>RNNs & LSTMs and Transformer</i>	Status
	Learning Outcomes	Completed
	• What are RNNs. • What is their use and they intricacies. • What are some of the problems RNNs have. • What are LSTM. • How they came up to solve the RNN problems. • How to combine LSTM and CNNs for image captioning. • What are the key concepts of BERT • How does the Attention mechanism work • Explain the details of the Transformer architecture	

Recurrent Neural Networks (RNNs) are a class of neural networks designed for processing sequential data by maintaining a hidden state that captures information about previous elements in the sequence. They are widely used in tasks like language modeling, speech recognition, and time series forecasting, where understanding context over time is crucial. RNNs work by feeding the output from the previous time step as input to the current one, allowing them to learn temporal dependencies. However, they suffer from challenges such as vanishing and exploding gradients, which make training over long sequences difficult, and struggle to retain long-term information, leading to performance issues in complex sequences. These limitations have led to the development of advanced variants like LSTMs and GRUs

Long Short-Term Memory (LSTM) networks are a special type of Recurrent Neural Network (RNN) designed to overcome the limitations of traditional RNNs, particularly the vanishing and exploding gradient problems. Introduced in the 1990s, LSTMs incorporate memory cells and a system of gates (input, forget, and output gates) that regulate the flow of information. This gating mechanism allows LSTMs to retain relevant information over long sequences and discard irrelevant details, making them capable of learning long-term dependencies more effectively than standard RNNs. As a result, LSTMs have been widely adopted in tasks like machine translation, speech recognition, and text generation, where context over long ranges is essential.

word embedding is a way of representing words as dense vectors of real numbers in a continuous vector space, where words with similar meanings or contexts are placed closer together. Unlike one-hot encoding, which treats words as discrete, unrelated tokens, embeddings capture semantic relationships and syntactic patterns between words by learning from large text corpora. These vector

representations make it easier for machine learning models to understand and process natural language.

Word2Vec: Introduced by Google, it uses models like Skip-gram and Continuous Bag of Words (CBOW) to learn word vectors based on predicting neighbouring words.

BERT (Bidirectional Encoder Representations from Transformers) is a powerful language model developed by Google that leverages the Transformer architecture to deeply understand language context through self-attention mechanisms. Unlike previous models that read text in a single direction, BERT processes text bidirectionally, capturing nuanced meaning from both left and right contexts. It is pretrained using two tasks: Masked Language Modelling (MLM), where some words are masked and predicted based on context, and Next Sentence Prediction (NSP), where the model learns sentence relationships. The attention mechanism allows BERT to weigh the importance of each word in a sentence relative to the others, enabling it to disambiguate meanings (e.g., distinguishing "bank" as a financial institution or riverbank). After pretraining on large corpora like Wikipedia, BERT can be fine-tuned for specific tasks such as question answering, sentiment analysis, or text classification, making it highly versatile and effective across a wide range of natural language processing applications.

Training a model for image captioning involves combining a convolutional neural network (CNN) to extract visual features from an image and a recurrent neural network (RNN), typically an LSTM, to generate a descriptive sentence. First, image-caption datasets like MS COCO are pre-processed by cleaning and tokenizing captions, adding start and end tokens, and converting words to sequences. Pretrained CNNs (like VGG16 or ResNet) are used as encoders to produce fixed-length feature vectors for images. These vectors, along with partial caption sequences, are input into an RNN decoder that learns to predict the next word in the sequence. During training, teacher forcing is used to guide the model, and during inference, captions are generated word by word starting from a "start" token until an "end" token is reached. More advanced methods use attention mechanisms and beam search to improve caption quality, and models are evaluated using metrics like BLEU or CIDEr

RNNs (Recurrent Neural Networks) are ideal for sequential data like time series or natural language. They process input one step at a time while maintaining hidden states to model temporal dependencies. Their strength is in handling sequences, but they struggle with long-term dependencies due to **vanishing gradients**, and training is slow due to their sequential nature.

CNNs (Convolutional Neural Networks) are best suited for spatial data, especially images and videos. They excel at extracting local patterns (edges, textures) using filters, and are highly efficient due to parameter sharing and local connectivity. Their limitation is that they don't handle sequential data or long-range dependencies well unless combined with RNNs or other architectures.

Transformers are powerful models designed for sequence modelling and have revolutionized NLP and increasingly vision tasks (like ViT). They use self-attention to capture global relationships in data, are highly parallelizable, and outperform RNNs on many tasks. However, they are computationally expensive and require large datasets and resources to train effectively.

ILO	Description: <i>Image Generation with GAN</i>	Status
	Learning Outcomes	Completed
	- Understand the background of how GAN works and how the development went towards GAN - Deep dream. - Style transfer. - fooling CNN - approach - GAN. - Applications of GAN for Image generation - general use cases - superresolution - Image to Image translation - Unpaired image translation	

Generative Adversarial Networks (GANs), consist of two neural networks a generator and a discriminator that compete in a zero-sum game. The generator creates synthetic data (like images) from random noise, aiming to mimic real data, while the discriminator tries to distinguish between real and generated data. Through this adversarial training, both networks improve: the generator becomes better at producing realistic outputs, and the discriminator sharpens its ability to detect fakes.

In GAN training, freezing the discriminator's weights when training the generator is essential to ensure that only the generator learns during its update phase. This technique prevents the discriminator from unintentionally updating its parameters when it is only supposed to evaluate the generator's outputs. During this phase, fake samples are generated and passed through the discriminator, but only the generator's weights are updated based on how well it fooled the discriminator. This is achieved by setting the discriminator's trainable flag to False before compiling the combined GAN model. Freezing thus maintains clear roles during training allowing stable adversarial learning where the discriminator judges and the generator improves based on feedback without disrupting the discriminator's learned knowledge.

Deep Dream is a CNN visualization technique that enhances patterns in existing images, while GANs are generative models that learn to create realistic new images from scratch. They serve different purposes but are both part of the broader field of deep generative and interpretive models.

Style transfer is a computer vision technique that involves recomposing an image in the style of another image while preserving its original content. It uses convolutional neural networks (CNNs) to separate and recombine content and style from two images. Content image: The image whose structure you want to keep (e.g., a photo of a landscape). Style image: The image whose artistic style you want to apply (e.g., a Van Gogh painting).

Fooling CNNs is about exploiting their vulnerability by carefully designing inputs by applying small perturbation or noise that mislead them highlighting a critical limitation of many deep learning models.

ILO	Description: <i>Fusion Stable Diffusion and VQA</i>	Status
	Learning Outcomes	Completed
	• Understand different fusion mechanisms, especially for neural networks • Explain how Stable Diffusion works • Understand the architecture for image captioning • Apply Stable Diffusion for image generation • Compare different approaches for image generation • Discuss issues coming with deep learning for image generation (Deepfakes, ...) • Understand the concepts of VQA	

Fusion mechanisms in neural networks refer to strategies used to combine multiple sources of information such as features from different modalities (e.g., image and text), layers, or models to improve overall performance. These mechanisms are critical in tasks like image captioning, visual question answering, and multimodal learning.

Here are the main types:

1. **Early Fusion:** Combines raw input features (e.g., image pixels + text tokens) before passing them into the neural network. It's simple but may miss complex relationships between modalities.
2. **Late Fusion:** Each modality is processed separately, and their outputs (e.g., predictions or features) are combined at a later stage. This is modular and easier to train but may not capture interactions deeply.
3. **Intermediate (Hybrid) Fusion:** Fusion occurs at multiple levels within the network. For example, combining mid-level image features with text embeddings. This is more expressive and allows deeper interaction.
4. **Attention-based Fusion:** Uses attention mechanisms to weigh the contribution of each modality dynamically. For example, in image captioning, the model attends to relevant image regions while generating each word.

Flow for image captioning:

Image Feature Extraction

- Use a pretrained CNN (e.g., VGG16, ResNet) to extract high-level feature vectors from the input image.
- This CNN acts as an encoder, transforming raw images into a compact, meaningful representation.

Sequence Modelling (Caption Generation)

- Pass the extracted image features as initial input to a language model, typically an RNN variant like an LSTM or GRU.
- The RNN acts as a decoder, generating the caption word-by-word based on image features and previously generated words.

Word Embedding Layer

- Convert input words (captions) into dense vector embeddings before feeding into the RNN to capture semantic meaning.

Training with Paired Data

- Train the model end-to-end on paired image-caption datasets, minimizing the difference between generated captions and ground-truth captions (e.g., using categorical cross-entropy loss).

Attention Mechanism (Optional but Common)

- Add an attention layer that lets the model focus on specific image regions for each generated word, improving relevance and detail.

Inference / Caption Generation

- At test time, generate captions by feeding image features to the decoder and sequentially predicting the next word until an end token is produced.
- Use techniques like greedy decoding or beam search for better caption quality.

Stable Diffusion:

Applying Stable Diffusion for image generation, you start with a pretrained diffusion model that learns to gradually denoise random noise into coherent images, conditioned on text prompts or other inputs. The process involves initializing with pure noise and then iteratively refining the image through a sequence of denoising steps guided by the model. Users typically provide a textual description (prompt), which the model encodes to steer the image synthesis towards the desired content. Stable Diffusion uses a latent space diffusion approach to efficiently generate high-quality images with fine details and creativity. Implementation often leverages libraries like Hugging Face's diffusers or specialized frameworks, where you load the pretrained model weights, supply your prompt, and run the denoising pipeline to produce novel images that match your input description. This approach enables powerful and flexible text-to-image generation, suitable for creative art, design, or content creation tasks.

Visual Question Answering (VQA) is a task where a model looks at an image and answers questions about it by combining image understanding (using CNNs) and language understanding (using RNNs or transformers). It uses fusion methods like attention to connect the visual and textual information, enabling the model to focus on relevant parts of the image based on the question. The goal is to teach AI to reason about both images and language together.

ILO	Description: <i>Reinforcement Learning & TinyML</i>	Status
	Learning Outcomes	Ongoing
	• Understand the major recent advances in Machine Learning and Deep Learning, in particular Reinforcement Learning with Policy Gradient and Q- Learning as well as Deep Q-learning. • Acquired familiarity with the latest research and advancements in the field of TinyML • Attained proficiency in training and deploying TinyML models on microcontrollers using TF-micro. • Mastered the optimization techniques for machine learning models tailored to resource-constrained devices like MCUs.	

